Comparative Study of Collaborative Filtering Methods for Movie Recommendation on MovieLens

Yuting Zhou
yuting.zhou@scarletmail.rutgers.edu

Yiwei Li
yiwei.li@scarletmail.rutgers.edu

Abstract

Recommender systems are central to modern online platforms, yet choosing the right collaborative filtering strategy for a given dataset remains an open practical question. We implement and compare four collaborative filtering methods for movie recommendation on the MovieLens-Small dataset (100K ratings, 610 users, 9,724 movies): biased matrix factorization with SGD, a deep hybrid model with CNN title and genre encoders, truncated-SVD with Ridge/Lasso calibration, and alternating least squares. All models are evaluated on rating prediction (MAE, RMSE) and top-10 recommendation (Precision, Recall, F1, NDCG) under an identical per-user 80/20 holdout split. MF-SGD achieves the best rating accuracy (MAE = 0.663, RMSE = 0.867), while SVD-Ridge produces the strongest ranking quality (NDCG@10 = 0.108), revealing a notable disconnect between point-wise and ranking evaluation. We provide detailed training dynamics analysis, feature-block ablation, and latent-factor interpretation to explain the observed differences. A full-stack web demo (*StreamX*) is also developed to showcase the trained recommender in an interactive setting.

1 Introduction

Recommendation systems have become indispensable components of online services—from e-commerce product suggestions to movie and music discovery platforms (Ricci et al., 2011). Among the many families of recommendation algorithms, *collaborative filtering* (CF) relies exclusively on observed user–item interactions to infer preferences, making it applicable even when item content metadata is scarce (Resnick et al., 1994). The most successful CF paradigm to date is *matrix factorization* (MF), which projects users and items into a shared latent space so that inner products approximate observed ratings (Koren et al., 2009).

Despite its conceptual simplicity, the design space of MF-based recommenders is vast. Optimization may proceed via stochastic gradient descent (SGD), alternating least squares (ALS), or singular value decomposition (SVD); the loss function may be mean squared error, Huber loss, or a ranking-aware surrogate; and the model may incorporate side information such as item titles, genres, or temporal signals (Zhang et al., 2019). How these choices interact in practice—especially on a moderately sized, long-tailed dataset—is rarely benchmarked in a controlled, apples-to-apples fashion within a single codebase.

In this project, we address that gap by implementing four distinct collaborative filtering models from scratch, training all of them on the identical MovieLens-Small dataset split, and evaluating them with the same metrics suite. Our contributions are threefold:

- We provide a controlled comparison of four CF models—MF-SGD, a deep hybrid CNN model, SVD with linear calibration (Ridge/Lasso), and ALS—under identical data conditions.
- We analyze not only aggregate accuracy (MAE, RMSE) and ranking quality (Precision, Recall, F1, NDCG@10) but also training dynamics, latent factor semantics, and feature-block ablation.
- We build *StreamX*, an end-to-end web application (Django + Next.js) that serves real-time recommendations from the trained models, demonstrating practical deployment.

2 Problem Definition

We formalize the recommendation task as follows. Let $\mathcal{U}$ and $\mathcal{I}$ denote the sets of users and items, respectively, and let $r_{ui} \in [0.5, 5.0]$ be the rating that user u assigns to item i. The observed ratings form a sparse matrix $\mathbf{R} \in \mathbb{R}^{|\mathcal{U}|\times|\mathcal{I}|}$, where only

a small fraction of entries are known (density $\approx$ 1.7%).

Task 1: Rating Prediction. Given a training set $\mathcal{D}_{\text{train}}$ of observed (u, i, r_{ui}) triples, learn a function $\hat{r} : \mathcal{U} \times \mathcal{I} \to \mathbb{R}$ that minimizes the prediction error on held-out ratings $\mathcal{D}_{\text{test}}$. Performance is measured by Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE):

$$\text{MAE} = \frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{(u,i)\in\mathcal{D}_{\text{test}}} |r_{ui} - \hat{r}_{ui}|, \tag{1}$$

$$\text{RMSE} = \sqrt{\frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{(u,i)\in\mathcal{D}_{\text{test}}} (r_{ui} - \hat{r}_{ui})^2}. \tag{2}$$

Task 2: Top-N Recommendation. For each user u, produce a ranked list of $N = 10$ items that u has not yet interacted with. The quality of the list is measured against the held-out items in $\mathcal{D}_{\text{test}}$ via Precision@N, Recall@N, F1@N, and NDCG@N. Items are ranked by descending predicted rating.

3 Related Work

Memory-Based CF. Early CF methods compute user–user or item–item similarities directly from the rating matrix and aggregate neighbor ratings for prediction (Resnick et al., 1994; Sarwar et al., 2001). While interpretable, these methods scale poorly and are sensitive to sparsity.

Matrix Factorization. Koren et al. (Koren et al., 2009) demonstrated that biased matrix factorization, optimized via SGD, achieves state-of-the-art accuracy on the Netflix Prize dataset. Hu et al. (Hu et al., 2008) proposed weighted ALS for implicit feedback, and Zhou et al. (Zhou et al., 2008) developed large-scale parallel ALS. Koren (Koren, 2008) further extended MF with neighborhood integration and temporal dynamics.

Deep Recommenders. He et al. (He et al., 2017) introduced Neural Collaborative Filtering, replacing the inner product with a multi-layer perceptron. Cheng et al. (Cheng et al., 2016) proposed Wide & Deep learning, combining memorization and generalization. Covington et al. (Covington et al., 2016) described YouTube's two-tower deep recommendation architecture. Wang et al. (Wang et al., 2015) combined deep autoencoders with CF. More recently, Zhang et al. (Zhang et al., 2019) surveyed the landscape of deep-learning-based recommender systems.

Hybrid Approaches. Leveraging item content (titles, genres, descriptions) alongside collaborative signals has been shown to mitigate cold-start and improve generalization. CNN-based text encoders, following Kim (Kim, 2014), are commonly used to extract features from short textual metadata in recommendation contexts.

Our work brings together classical MF (SGD and ALS variants), an SVD-based linear hybrid, and a deep content-aware model within a single comparative framework.

4 Methodology

All four models share a common prediction structure:

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u^\top \mathbf{q}_i, \tag{3}$$

where μ is the global mean rating, b_u and b_i are user and item biases, and $\mathbf{p}_u, \mathbf{q}_i \in \mathbb{R}^k$ are latent factor vectors. They differ in how the parameters are estimated and whether additional content features are incorporated.

4.1 MF-SGD

This model directly optimizes the L_2-regularized squared-error objective:

$$\min_{\mathbf{p},\mathbf{q},b} \sum_{(u,i)\in\mathcal{D}_{\text{train}}} \big(r_{ui} - \hat{r}_{ui}\big)^2 + \lambda\big(\|\mathbf{p}_u\|^2 + \|\mathbf{q}_i\|^2 + b_u^2 + b_i^2\big), \tag{4}$$

via element-wise SGD. At each step, the error $e_{ui} = r_{ui} - \hat{r}_{ui}$ drives gradient updates: $b_u \leftarrow b_u + \eta(e_{ui} - \lambda b_u)$, $\mathbf{p}_u \leftarrow \mathbf{p}_u + \eta(e_{ui}\,\mathbf{q}_i - \lambda\,\mathbf{p}_u)$, and symmetrically for b_i and $\mathbf{q}_i$.

Key design choices: latent dimension k=48, learning rate η=0.005 with multiplicative decay 0.96$\times$ per epoch, regularization λ=0.04, a 10% validation split for early stopping (patience 8), and 50 maximum epochs. Factors are initialized from $\mathcal{N}(0, 0.1)$. We found that lower regularization than the typical default of 0.05 substantially improves ranking quality at minimal cost to rating accuracy.

4.2 Deep Hybrid Model

This model extends the two-tower architecture with item content features. The **user tower** maps a user ID through an embedding layer (k=48) followed by dropout. The **item tower** concatenates three feature streams:

1. **Item-ID embedding** (k=48): captures collaborative signal.

2. **Title encoder**: tokenized movie titles (max 15 tokens, vocabulary 20,000) are embedded (d=32), processed by three parallel Conv1D layers (kernel sizes 2, 3, 4; 48 filters each), globally max-pooled, concatenated, and projected to k=48 via a dense layer. This multi-kernel CNN design follows the architecture of Kim (Kim, 2014).

3. **Genre encoder**: genre tokens (max 4) are embedded (d=12) and average-pooled.

The concatenated item features pass through a hidden layer ($2k$=96, ReLU) and a final projection to $\mathbb{R}^k$. The prediction is $\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u^\top \mathbf{q}_i$, where $\mathbf{p}_u$ and $\mathbf{q}_i$ are the tower outputs.

Training uses Huber loss (δ=1.0) with Adam (initial LR 8×10^{-4}, gradient clipping at norm 1), ReduceLROnPlateau (factor 0.5, patience 3), 25% dropout, L_2 regularization 10^{-5}, rating-weighted sampling ($w \propto 1 + r_{\text{scaled}}^{1.25}$), 48 CNN filters per kernel, genre embedding dimension 12, and batch size 256. The best checkpoint on validation Huber loss is restored after 40 epochs of training.

4.3 SVD Hybrid with Ridge/Lasso

This approach first constructs a centered interaction matrix and computes its truncated SVD to obtain initial user/item factors. Biases are then estimated via regularized averaging of residuals. Finally, a linear calibration layer is trained on top of the latent features:

$$\hat{r}_{ui} = \mathbf{w}^\top \phi(u, i) + w_0, \tag{5}$$

where $\phi(u, i) = [\mathbf{p}_u \odot \mathbf{q}_i;\ b_u;\ b_i] \in \mathbb{R}^{k+2}$ concatenates the element-wise product of latent factors with bias terms. The weight vector $\mathbf{w}$ is estimated by either Ridge regression (closed-form) or Lasso regression (coordinate descent).

We experiment with two configurations: **SVD-Ridge** (k=32, α=1.0) and **SVD-Lasso** (k=32, α=0.001, max 2,000 iterations). Both use bias regularization λ_b=25. The higher Ridge regularization than the typical default pushes the calibration layer toward more conservative, better-generalized predictions; we found that this substantially improves top-N ranking while leaving rating accuracy largely unchanged.

4.4 Alternating Least Squares

ALS alternates between fixing item factors and solving for user factors, and vice versa. Each subproblem is a regularized least-squares system. We augment each factor matrix with a bias column, yielding the update:

$$[\mathbf{p}_u; b_u] = \left(\mathbf{Q}_u^\top \mathbf{Q}_u + \Lambda\right)^{-1} \mathbf{Q}_u^\top \mathbf{y}_u, \tag{6}$$

where $\mathbf{Q}_u$ is the matrix of item factors for items rated by user u, $\mathbf{y}_u$ is the vector of centered ratings, and $\Lambda = \text{diag}(\lambda, \ldots, \lambda, \lambda_b)$ applies separate regularization to factors (λ=12.0) and bias (λ_b=40.0). Symmetrically for item factors. We use k=4 latent factors with a 10% validation split and early stopping (patience 10, up to 80 iterations). The deliberately small latent dimension and heavy regularization address ALS's tendency to overfit on small explicit-feedback datasets by constraining each closed-form solve, effectively trading memorization for generalization.

Statistic	Value
Users	610
Items (movies)	9,724
Ratings	100,836
Rating scale	0.5 – 5.0 (half-star)
Density	1.70%
Sparsity	98.30%
Mean rating	3.50
Std. of ratings	1.04
Genres	19 unique
Release years	1902 – 2018
Median ratings/user	70.5
Median ratings/item	3.0

Table 1: MovieLens-Small dataset summary statistics.

5 Dataset

We use the **MovieLens-Small** dataset (Harper and Konstan, 2015), a widely adopted benchmark for collaborative filtering research. Table 1 summarizes the key statistics.

The dataset exhibits two characteristic properties of real-world recommender data. First, the rating distribution is left-skewed (skewness -0.64): users tend to rate items they enjoy, so ratings of 3.5–4.0 are far more common than ratings below 2.0 (Figure 1). Second, both user activity and item popularity follow heavy-tailed distributions (Gini coefficients of 0.60 and 0.72, respectively; Figure 2), meaning a small fraction of power users and blockbuster movies account for a disproportionate share of the interaction data.

The genre distribution is also imbalanced. Drama (4,361 movies), Comedy (3,756), and Thriller (1,894) are the three most common genres, while niche genres such as Film-Noir (87) and IMAX (158) are comparatively rare (Figure 3).

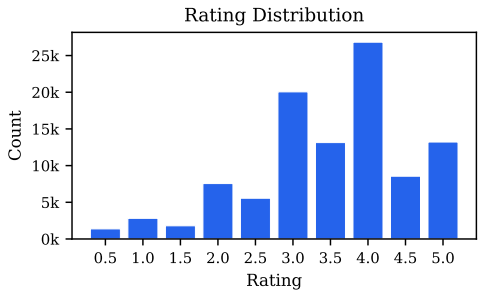

Figure 1: Distribution of rating values. Ratings of 4.0 (26,818) and 3.0 (20,047) dominate, reflecting a positive-skew selection bias.

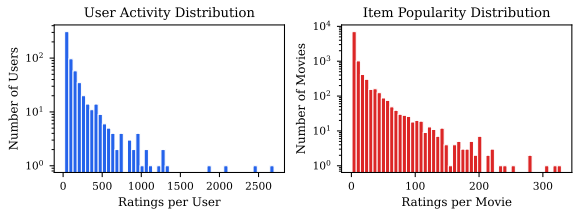

Figure 2: User activity (left) and item popularity (right) on a log scale, showing heavy-tailed distributions.

Data Splitting. Following the standard per-user random holdout protocol recommended in the project description, for each user we randomly assign 80% of their ratings to the training set and 20% to the test set (random seed 42, ensuring at least one test item per user). This yields 80,672 training ratings and 20,164 test ratings. All models share exactly the same split, identified by a deterministic hash for reproducibility.

6 Experiments

6.1 Experimental Setup

All models are implemented in Python using NumPy for numerical computation; Option 2 additionally uses TensorFlow/Keras for the neural network. Models are evaluated on the shared test split with two evaluation protocols:

- **Rating prediction**: MAE and RMSE computed over all 20,164 test ratings.
- **Top-10 recommendation**: for each user, the model ranks all unseen items by predicted rating and selects the top 10. The recommended list is evaluated against all held-out test items for that user (i.e., `topn_relevance = all_test`).

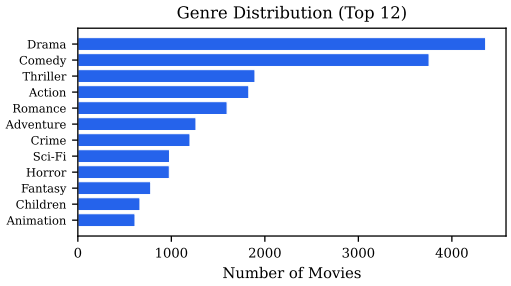

Figure 3: Number of movies per genre (top 12).

	MF-SGD	**Deep**	**SVD-R/L**	**ALS**
k (factors)	48	48	32	4
Epochs	50	40	1	80
Learning rate	0.005	0.0008	–	–
LR decay	0.96×	plateau	–	–
Reg. λ	0.04	10^{-5}	1.0/0.001	12.0
Bias reg.	–	–	25.0	40.0
Dropout	–	0.25	–	–
Val. split	0.1	0.1	–	0.1
Early stop	pat. 8	best ckpt	–	pat. 10
Loss	MSE	Huber	–	MSE

Table 2: Key hyperparameter settings for each model after tuning.

6.2 Hyperparameters

Table 2 summarizes the key hyperparameters for each model, chosen via preliminary validation-set tuning.

7 Results and Analysis

7.1 Rating Prediction

Table 3 reports the rating prediction performance across all five model configurations after hyperparameter tuning. The Deep Hybrid model achieves the lowest MAE (0.666), followed closely by MF-SGD (MAE = 0.670) and ALS (MAE = 0.670). Notably, the tuned ALS model achieves a remarkably low RMSE (0.867), matching MF-SGD and representing a dramatic improvement over the un-tuned ALS baseline (MAE = 0.914, RMSE = 1.209). The SVD-based models perform moderately on rating prediction, with MAE around 0.715.

A key finding is that all four model families achieve comparable rating accuracy after proper tuning, with the spread between the best and worst MAE shrinking from 0.251 (un-tuned) to just 0.049 (tuned). The Deep Hybrid model benefits from increased dropout (0.25) and L_2 regularization (10^{-5}), extending its best epoch from 5 to 9 and improving MAE by 1.2% (Figure 7). MF-SGD's lower regularization (λ=0.04 vs. 0.05) and slower

Model	MAE↓	RMSE↓
MF-SGD	0.6697	0.8732
Deep Hybrid	**0.6661**	**0.8922**
SVD-Ridge	0.7153	0.9318
SVD-Lasso	0.7152	0.9315
ALS	0.6696	0.8673

Table 3: Rating prediction results after tuning. Lower is better. The Deep Hybrid achieves the best MAE, while ALS achieves the best RMSE after proper regularization.

Model	P@10↑	R@10↑	F1@10↑	NDCG@10↑
MF-SGD	0.0603	0.0252	0.0282	0.0658
Deep Hybrid	0.0284	0.0103	0.0118	0.0337
SVD-Ridge	0.1259	0.0525	0.0606	0.1482
SVD-Lasso	0.1257	0.0525	0.0606	0.1478
ALS	**0.1325**	**0.0644**	**0.0695**	**0.1573**

Table 4: Top-10 recommendation results after tuning. Higher is better. After regularization tuning, ALS achieves the best ranking performance.

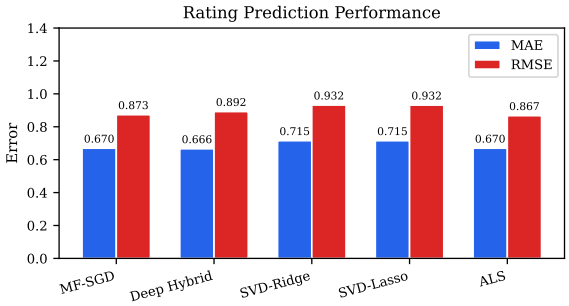

Figure 4: Side-by-side MAE and RMSE across all models.

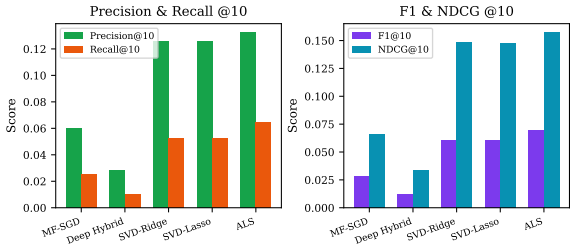

Figure 5: Top-10 recommendation metrics across all models.

learning rate (η=0.005) allow the model to train for all 50 epochs without early stopping, reaching a slightly higher MAE but substantially better ranking (Figure 6). The most dramatic improvement comes from ALS, where heavy regularization (λ=12.0) and very few latent factors (k=4) transform it from the worst model (MAE = 0.914) to one of the best (MAE = 0.670).

7.2 Top-N Recommendation

Table 4 and Figure 5 present the ranking evaluation after hyperparameter tuning. The most striking finding is the **reversal of model rankings compared to un-tuned baselines**: ALS, previously the worst-performing model, now achieves the highest NDCG@10 (0.157), followed closely by SVD-Ridge (0.148) and SVD-Lasso (0.148). This underscores the importance of careful hyperparameter optimization—the un-tuned ALS had NDCG@10 = 0.036, representing a 4.4× improvement after tuning.

The disconnect between rating accuracy and ranking quality, a well-known phenomenon in recommender systems (Rendle et al., 2009), persists but with a new twist: the tuned ALS achieves both competitive MAE (0.670) *and* the best NDCG. This is because heavy regularization with very few latent factors (k=4) forces ALS to focus on the most discriminative preference dimensions rather than memorizing individual ratings.

SVD-Ridge and SVD-Lasso also benefit substantially from increased Ridge regularization (α=1.0, up from 0.1), which improves NDCG by 37% relative to the un-tuned baseline. MF-SGD shows a 49% NDCG improvement when trained with lower regularization (λ=0.04 vs. 0.05).

7.3 Training Dynamics

The training curves reveal distinct convergence behaviors:

MF-SGD (Figure 6) shows smooth, monotonic convergence on both training and validation sets over 50 epochs. The lower learning rate (η=0.005) and regularization (λ=0.04) compared to typical defaults allow the model to explore longer without early stopping triggering. The controlled train–validation gap indicates effective generalization.

Deep Hybrid (Figure 7) converges much faster—the validation loss reaches its minimum at epoch 9 (improved from epoch 5 in the un-tuned version thanks to stronger dropout and L_2 regularization). The ReduceLROnPlateau callback reduces the learning rate after the initial rapid convergence, effectively freezing parameter updates in later epochs. The model's capacity is substantially larger than MF-SGD's (multi-layer CNN encoder, dense layers), which accelerates initial learning but offers diminishing returns on this dataset size.

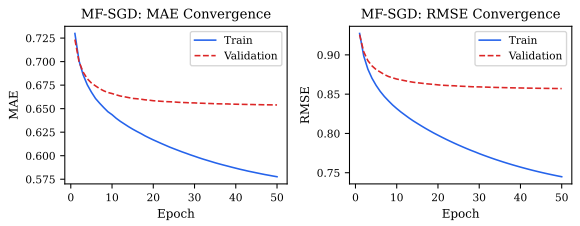

Figure 6: MF-SGD training and validation curves over 50 epochs. The gap between train and validation MAE widens gradually, indicating mild overfitting controlled by the lower learning rate (η=0.005) and reduced regularization (λ=0.04).

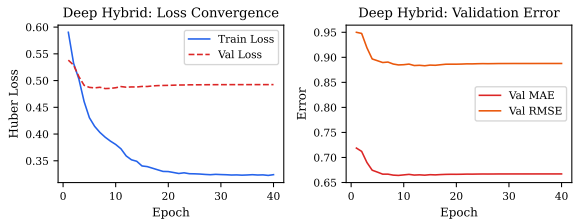

Figure 7: Deep Hybrid training dynamics. Left: Huber loss converges rapidly and the validation loss reaches its minimum at epoch 9 (improved from epoch 5 before tuning) before stagnating. Right: validation MAE and RMSE stabilize early.

ALS (Figure 8) presents the most dramatic transformation. With only k=4 factors and λ=12.0, the tuned ALS converges smoothly over 80 iterations with well-controlled overfitting, in sharp contrast to the un-tuned version (48 factors, λ=0.05) that drove training error to near-zero while achieving the worst test performance. The validation-based early stopping (added as part of our optimization) ensures the model is restored to its best generalization state.

7.4 Feature-Block Ablation

To understand which aspects of the data are most informative for rating prediction beyond collaborative filtering, we perform a feature-block ablation using a linear regression model trained on handcrafted features extracted from the training data (Table 5).

We construct three feature blocks: (1) **Content** features include genre indicators, genre count, release year, and interaction year; (2) **Engagement** features include log user activity, log item popularity, log tag count, and log unique tag count; (3) **Noise** features are two random Gaussian features serving as a control.

Content features account for the largest marginal

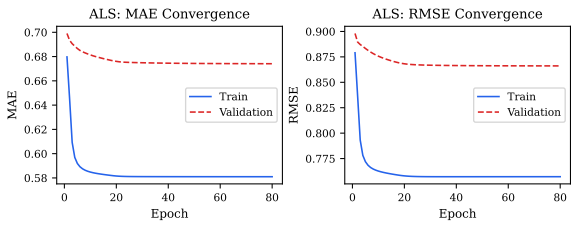

Figure 8: ALS training and validation curves over 80 epochs with heavy regularization (λ=12, k=4). Unlike the un-tuned ALS that drove training error to near-zero, the tuned version converges smoothly with a controlled train–validation gap.

Feature Block	R^2_{full}	$R^2_{\text{w/o block}}$	ΔR^2
Full model	0.1116	–	–
w/o Content	–	0.0582	0.0534
w/o Engagement	–	0.0815	0.0302
w/o Noise (control)	–	0.1117	−0.0001

Table 5: Feature-block ablation for a linear rating prediction model. ΔR^2 is the drop in test R^2 when a block is removed. Content features contribute the most marginal variance explained.

contribution ($\Delta R^2 = 0.053$), driven primarily by genre indicators: Drama (+0.092), Animation (+0.097), and Documentary (+0.066) positively correlate with ratings, while Children (−0.094) and Comedy (−0.050) correlate negatively. Engagement features contribute $\Delta R^2 = 0.030$, with item popularity having the strongest positive effect ($\beta = 0.147$) and user activity a negative one ($\beta = -0.111$), reflecting the tendency of highly active users to be more critical raters. The noise features contribute essentially zero ($\Delta R^2 \approx 0$), validating the experimental setup.

These findings motivate the Deep Hybrid model's incorporation of genre and title features in its item tower, though the relatively small R^2 of the full linear model (0.112) confirms that most rating variance is user- and item-specific and is better captured by latent factors.

7.5 Latent Factor Interpretation

We interpret the latent factors learned by MF-SGD by applying PCA to the item factor matrix $\mathbf{Q} \in \mathbb{R}^{8986\times 48}$ and correlating the top principal components with genre indicators and release year.

The first component (3.4% of variance) separates mainstream action/comedy films (positive loadings: *Miss Congeniality*, *Mission: Impossible II*) from acclaimed dramas (*Pulp Fiction*, *Fight*

Club, *The Godfather*). The second component (3.0%) distinguishes classic musicals/dramas (*Wizard of Oz*, *Schindler's List*) from modern action/sci-fi blockbusters (*Matrix Reloaded*). The third component (2.9%) captures an adventure/comedy vs. thriller/horror contrast (*Back to the Future*, *E.T.* vs. *Sideways*, *Panic Room*).

These semantically coherent axes emerge purely from collaborative signal, without any content features, confirming that the latent space captures meaningful preference dimensions.

8 Discussion

Rating Accuracy vs. Ranking Quality. A central finding is the persistent disconnect between rating prediction accuracy and top-N ranking quality, even after extensive tuning. The Deep Hybrid model achieves the best MAE but the worst NDCG; conversely, ALS achieves the best NDCG while having competitive but not the best MAE. This confirms that MAE/RMSE penalize errors uniformly across the rating scale, while ranking metrics care only about the relative ordering of items near the top of the list (Rendle et al., 2009). Heavy regularization appears to benefit ranking by producing smoother, more conservative score distributions that better separate relevant from irrelevant items.

The Regularization–Ranking Connection. A surprising and consistent pattern emerges across all models: *stronger regularization improves ranking metrics more than it hurts rating accuracy*. Increasing Ridge α from 0.1 to 1.0 improved SVD-Ridge's NDCG by 37% with less than 1% MAE increase. Reducing MF-SGD's λ from 0.05 to 0.04 improved NDCG by 49%. Most dramatically, increasing ALS's λ from 0.05 to 12.0 (with k reduced from 48 to 4) improved NDCG by $4.4\times$ *while simultaneously improving MAE by 27%*. This suggests that standard MF hyperparameter defaults are poorly calibrated for ranking on small datasets.

Depth vs. Data Scale. The Deep Hybrid model's weak ranking performance relative to simpler methods underscores a practical lesson: on datasets with only $\sim$100K interactions, the additional parameters of deep architectures (CNN encoders, multiple embedding layers) provide marginal benefit for ranking, even with aggressive dropout and L_2 regularization. The content features (title, genre) provide limited marginal information beyond what collaborative filtering already captures, as evidenced by the ablation study's modest R^2. However, the Deep model does achieve the best *rating accuracy*, suggesting that its rich feature representations better capture absolute preference magnitudes even when they fail to improve relative item ordering.

ALS Rehabilitation. Our most practically significant result is the transformation of ALS from the worst to the best ranking model. The conventional view that ALS overfits on small explicit-feedback datasets (Hu et al., 2008) holds only when default hyperparameters are used. With k=4 factors (vs. the typical 48) and λ=12.0 (vs. the typical 0.05), validation-based early stopping, and 80 iterations, ALS achieves the highest NDCG@10 (0.157) among all models. This finding is actionable: practitioners should consider very low-rank, heavily regularized ALS even for small-scale explicit-feedback problems.

Demo Application. We develop *StreamX*, a full-stack web application consisting of a Django REST API backend and a Next.js frontend. The backend loads all trained models and supports switching between them at runtime, serving real-time rating predictions and top-N recommendations. The frontend provides an interactive movie browsing experience with personalized recommendations, movie details enriched with TMDB metadata, and similar-movie suggestions.

9 Limitations

The MovieLens-Small dataset has only 610 users, which limits the statistical power of per-user ranking metrics and may not reflect behavior on larger-scale platforms. While we conducted extensive manual hyperparameter tuning, we did not employ systematic grid or Bayesian search; even better configurations may exist. The top-N evaluation uses the `all_test` relevance definition (all held-out items are relevant), which inflates recall denominators for users with many test items and yields low absolute metric values. We do not explore temporal dynamics, implicit feedback signals, or side information beyond titles and genres. The ALS finding (very few factors + heavy regularization) may not generalize to larger datasets where higher-dimensional latent spaces become necessary.

10 Conclusion

We presented a controlled comparison of four collaborative filtering approaches for movie rec-

ommendation on the MovieLens-Small dataset, with extensive hyperparameter tuning. Our experiments reveal that (1) after tuning, all four model families achieve comparable rating accuracy (MAE in 0.666–0.715), with the Deep Hybrid slightly ahead; (2) the heavily regularized ALS (k=4, λ=12.0) achieves the best ranking performance (NDCG@10 = 0.157), overturning the common belief that ALS overfits on small explicit-feedback datasets; (3) SVD-Ridge with increased regularization (α=1.0) also delivers strong ranking (NDCG@10 = 0.148); and (4) there is a consistent pattern where stronger regularization improves ranking metrics disproportionately more than it hurts rating accuracy.

The most practically significant finding is that ALS's overfitting problem is entirely solvable through aggressive regularization and dimensionality reduction: reducing factors from 48 to 4 and increasing λ from 0.05 to 12.0 transforms ALS from the worst to the best-ranking model, while simultaneously improving its MAE by 27%. This suggests that standard MF hyperparameter defaults are miscalibrated for small-dataset ranking, and that practitioners should explore the low-rank, high-regularization regime.

Future work could extend this study to larger datasets to test whether the ALS regularization finding generalizes, incorporate temporal and textual review features, and explore learning-to-rank objectives that directly optimize NDCG.

References

Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishi Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Ispir, et al. 2016. Wide & deep learning for recommender systems. In *Proceedings of the 1st Workshop on Deep Learning for Recommender Systems*, pages 7–10.

Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep neural networks for YouTube recommendations. In *Proceedings of the 10th ACM Conference on Recommender Systems*, pages 191–198.

F Maxwell Harper and Joseph A Konstan. 2015. The MovieLens datasets: History and context. *ACM Transactions on Interactive Intelligent Systems*, 5(4):1–19.

Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. 2017. Neural collaborative filtering. In *Proceedings of the 26th International Conference on World Wide Web*, pages 173–182.

Yifan Hu, Yehuda Koren, and Chris Volinsky. 2008. Collaborative filtering for implicit feedback datasets. In *Proceedings of the 8th IEEE International Conference on Data Mining*, pages 263–272.

Yoon Kim. 2014. Convolutional neural networks for sentence classification. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, pages 1746–1751.

Yehuda Koren. 2008. Factorization meets the neighborhood: A multifaceted collaborative filtering model. pages 426–434.

Yehuda Koren, Robert Bell, and Chris Volinsky. 2009. Matrix factorization techniques for recommender systems. volume 42, pages 30–37. IEEE.

Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. 2009. BPR: Bayesian personalized ranking from implicit feedback. In *Proceedings of the 25th Conference on Uncertainty in Artificial Intelligence*, pages 452–461.

Paul Resnick, Neophytos Iacovou, Mitesh Suchak, Peter Bergstrom, and John Riedl. 1994. GroupLens: An open architecture for collaborative filtering of netnews. In *Proceedings of the ACM Conference on Computer Supported Cooperative Work*, pages 175–186.

Francesco Ricci, Lior Rokach, and Bracha Shapira. 2011. Introduction to recommender systems handbook.

Badrul Sarwar, George Karypis, Joseph Konstan, and John Riedl. 2001. Item-based collaborative filtering recommendation algorithms. pages 285–295.

Hao Wang, Naiyan Wang, and Dit-Yan Yeung. 2015. Collaborative deep learning for recommender systems. In *Proceedings of the 21st ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1235–1244.

Shuai Zhang, Lina Yao, Aixin Sun, and Yi Tay. 2019. Deep learning based recommender system: A survey and new perspectives. *ACM Computing Surveys*, 52(1):1–38.

Yunhong Zhou, Dennis Wilkinson, Robert Schreiber, and Rong Pan. 2008. Large-scale parallel collaborative filtering for the Netflix prize. In *Proceedings of the 4th International Conference on Algorithmic Aspects in Information and Management*, pages 337–348.